

By Simran Kaur Bindra

1. Quick Reference Table

$O(1)$ | Constant | $T(n) = c$

$O(\log n)$ | Logarithmic | $T(n) = c \log(n)$

$O(n)$ | Linear | $T(n) = c*n$

$O(n^2)$ | Quadratic | $T(n) = c*n^2$

$O(2^n)$ | Exponential | $T(n) = c*2^n$

2. Time Complexity Examples

$O(1)$ Constant Time

```
arr = [10, 20, 30, 40, 50]
```

```
print(arr[2])
```

Derivation: Constant time, independent of n .

$O(n)$ Linear Time

```
for i in range(len(arr)):
```

```
    print(arr[i])
```

Derivation: Runs n times $\rightarrow$ linear.

$O(\log n)$ Logarithmic Time

```
n = 1024
```

```
while n > 1:
```

```
    n = n // 2
```

Derivation: Each step halves input $\rightarrow \log(n)$ steps.

$O(n^2)$ Quadratic Time

```
for i in range(n):
```

```
    for j in range(n):
```

```
        print(i, j)
```

Derivation: $n * n = n^2$ operations.

3. Space Complexity Examples

sum = a + b -> $O(1)$

arr = [i for i in range(n)] -> $O(n)$

Recursive factorial -> $O(n)$

Matrix = [[0]*n for _ in range(n)] -> $O(n^2)$

4. Algorithm vs Complexity

Linear Search: $O(n)$

Binary Search: $O(\log n)$

Bubble Sort: $O(n^2)$

Merge Sort: $O(n \log n)$

Quick Sort: $O(n \log n)$

Selection Sort: $O(n^2)$

5. Doubling Table

n | $O(1)$ | $O(\log n)$ | $O(n)$ | $O(n^2)$ | $O(2^n)$

2 | 1 | 1 | 2 | 4 | 4

4 | 1 | 2 | 4 | 16 | 16

8 | 1 | 3 | 8 | 64 | 256

16 | 1 | 4 | 16 | 256 | 65536

Remember:

Constant < Logarithmic < Linear < Quadratic < Exponential